

Solist-AI™

加速度センサを使った異常検知と

AI ライブラリ使用方法

本ドキュメントは、加速度センサを使った異常検知サンプルファームウェアと AI ライブラリ的使用方法について記載しています

更新履歴

日付	バージョン	内容
2025/04/18	001	初版発行
2026/03/23	002	2.3 センサ接続の注意点を追加 2.4 サンプルファームウェアの仕様を追加 Solist-AI™ Scope library の仕様を削除 3.1.3 パラメータ追加

目次

1.	概要	5
1.1.	開発環境の構成	5
1.2.	開発環境のセットアップ	5
1.2.1.	ファームウェア開発環境のセットアップ	5
1.2.2.	モニタツール Solist-AI™ Scope のセットアップ (Optional)	6
2.	加速度センサを使った異常検知サンプルファームウェア	7
2.1.	ブロック図	7
2.2.	端子の接続	8
2.3.	KXTJ3-1057-EVK-001 の注意点	8
2.4.	フォルダ構成	9
2.5.	サンプルファームウェア仕様	10
2.5.1.	概略フロー	10
2.5.2.	異常検知	10
2.5.3.	状態遷移	11
2.5.4.	LED	11
2.5.5.	センサの設定	11
2.6.	Solist-AI™ Scope を使った学習・推論の確認	12
2.7.	加速度センサの固定	14
2.8.	Demo board library	15
2.8.1.	API 仕様	15
2.9.	Sensor Driver library	15
2.9.1.	API 仕様	15
3.	Solist-AI™ library	16
3.1.	AI library	16
3.1.1.	AI への入出力データ数	16
3.1.1.	AI モデルの設定	17
3.1.2.	主な API の仕様	18
3.1.3.	API 仕様	18
3.1.4.	Solist-AI™ Sim で作成したモデルのインポート	25
3.1.5.	初期学習時のフロー	26
3.1.6.	推論フロー	27
3.1.7.	逐次学習フロー	28
3.1.8.	モデルをインポート時の推論フロー	29
3.2.	FFT library	30
3.2.1.	仕様	30
3.2.2.	API 仕様	30
3.2.3.	Usage	31
3.3.	オンデバイス学習と重みデータ	33
3.3.1.	3 層ニューラルネットワーク	33
3.3.2.	異常検知	33
3.3.3.	重みデータのメモリ使用量	34
3.3.4.	ESN (Echo State Network)	35
3.3.5.	bfloat16	36

4. 商標について 37

1. 概要

Solist-AI™を使った異常検知のサンプルファームウェアと、Solist-AI™ライブラリの使い方について説明します。

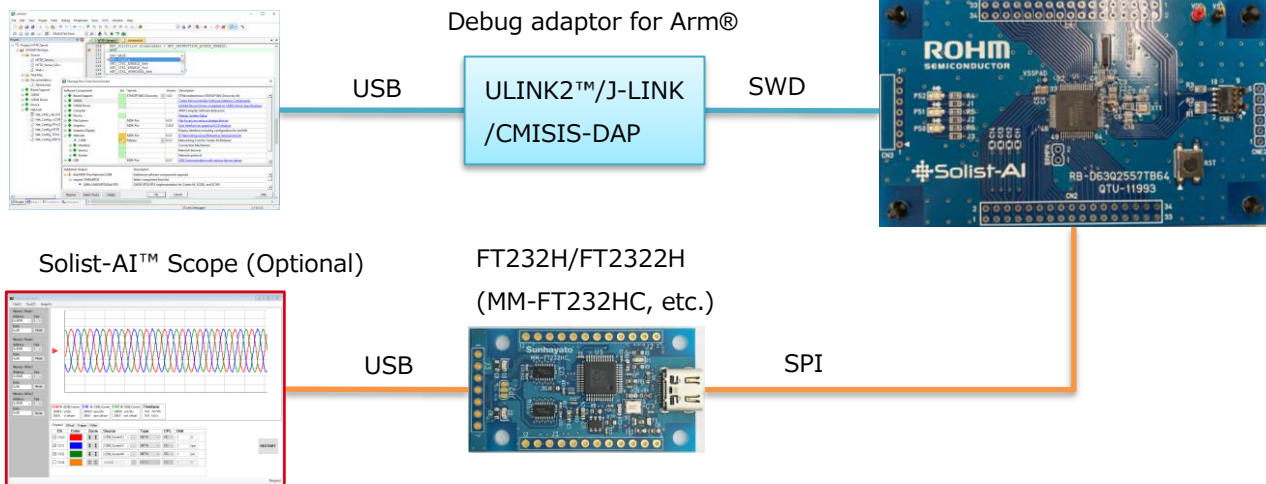
1.1. 開発環境の構成

接続については、各ボードのマニュアルをご覧ください。

LEXIDE-Ω

Arm® Keil® μVision®

Solist-AI™ MCU (ML63Q25xx) board



1.2. 開発環境のセットアップ

1.2.1. ファームウェア開発環境のセットアップ

いずれかの Arm®のファームウェア開発環境をインストールしてください。ここでは LEXIDE-ΩとμVision® 5 について説明します。

デバッグには、Arm® Keil® ULINK2™, J-LINK, CMSIS-DAP のいずれかのデバッグアダプタ（別売り）が必要です。

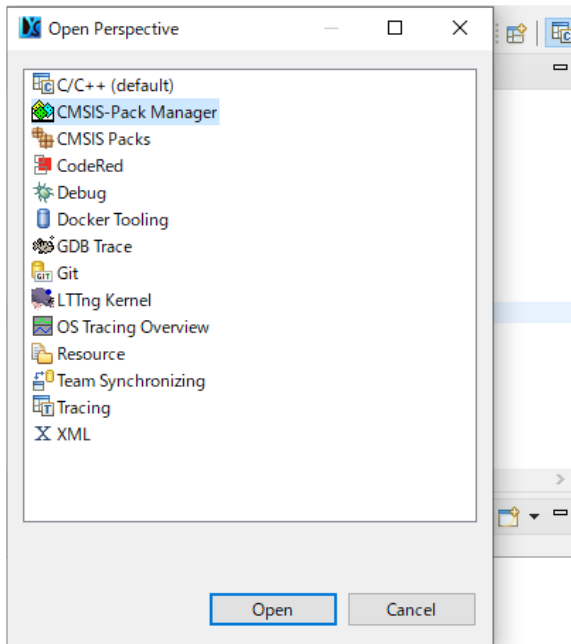
（LEXIDE-Ωは、ULINK2™に対応していません）

1. LEXIDE-Ω

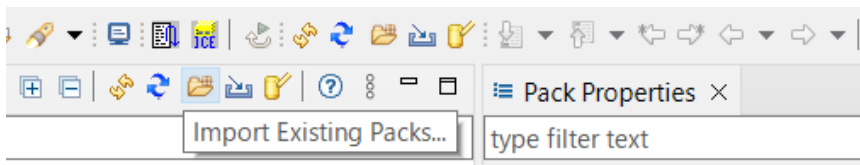
ロームのマイコン Web サイトより、インストーラをダウンロードし、インストール。

ロームのマイコン Web サイトより、“ML63Q2500 Software Pack”をダウンロードし、zip ファイルを展開。

LEXIDE-Ωの右上端の から“Open Perspective” ダイアログを開き、“CMSIS-Pack Manager”を起動。



“CMSIS-Pack Manager”の“Import Existing Packs...”から、ROHM.ML63Q25**_DFP.*.*.pack をインストール。

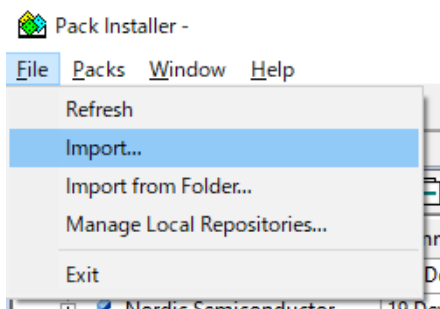


2. Keil® μVision® 5

Arm®の Web サイトよりμVision®をダウンロードし、インストール。

ロームのマイコン Web サイトより、“ML63Q2500 Software Pack”をダウンロードし、zip ファイルを展開。

μVision®から Pack Installer を起動し、File – Import から ROHM.ML63Q25**_DFP.*.*.pack をインストール。



1.2.2. モニタツール Solist-AI™ Scope のセットアップ (Optional)

Solist-AI™ Scope ユーザーズガイドに従ってインストールしてください。

Solist-AI™ Scope を使うには、FT232H/FT2232H 搭載ボード (MM-FT232HC など) (別売り) が必要です。

2. 加速度センサを使った異常検知サンプルファームウェア

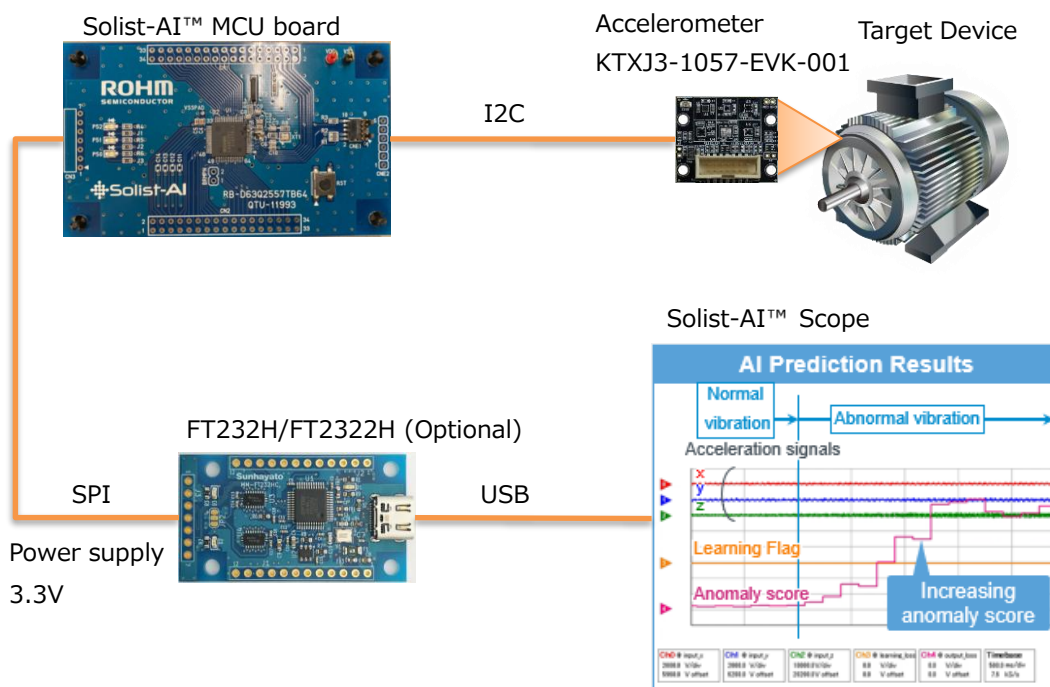
加速度センサを使った異常検知のサンプルファームウェアを用意しています。

Arm® Keil® μVision®では samples/Solist-AI/AnomalyDetectionDemo/AnomalyDetectionDemo.uvprojx を開いてください。
LEXIDE-Ωでは、samples/Solist-AI/AnomalyDetectionDemo フォルダをワークスペースに追加してください。

Solist-AI™ 搭載ボードとしては RB-D63Q25xx（リファレンスボード）、加速度センサボードは KXTJ3-1057-EVK-001 を対象としています。

2.1. ブロック図

Solist-AI™ MCU board からセンサに電源を供給する場合、ボードの電源電圧がセンサの動作電圧の範囲内であることを確認してください。



2.2. 端子の接続

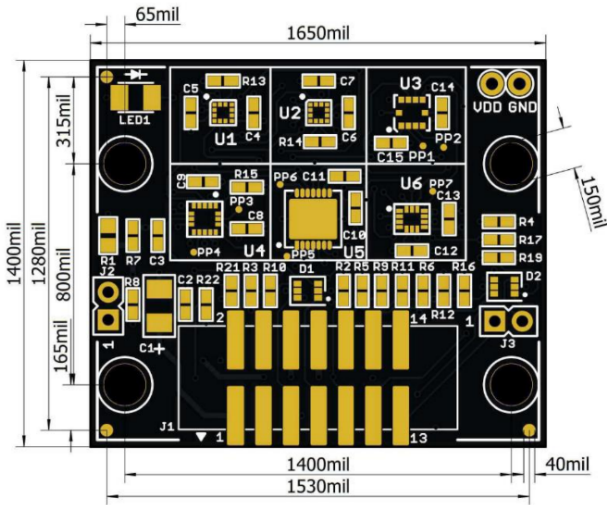
サンプルファームウェアでは、加速度センサを I2C で接続しています。
I2C でセンサを接続する場合、SDA, SCL 端子がプルアップされていることを確認してください。

Pin	Function	I/O	PU/PD	Active	Name	Description
P50	GPIO	O		L	LED1	
P51	GPIO	O		L	LED2	
P52	GPIO	O		L	LED3	
P40	SCKF0	O			SCK0	(Accelerometer SPI)
P41	SOUTF0	O			MOSI0	(Accelerometer SPI)
P42	SINF0	I			MISO0	(Accelerometer SPI)
P43	SSNF0	O			CS0	(Accelerometer SPI)
P60	SCKF1	I			SCK1	Solist-AI Scope SPI
P61	SOUTF1	O			MOSI1	Solist-AI Scope SPI
P62	SINF1	I			MISO1	Solist-AI Scope SPI
P63	SSNF1	I			CS1	Solist-AI Scope SPI
P74	SDAF0	O			SDA0	Accelerometer I2C
P73	SCLF0	O			SCL0	Accelerometer I2C

2.3. KXTJ3-1057-EVK-001 の注意点

- 本サンプルファームウェアを動かすには、以下を確認してください。
- I2C 通信ライン (R2, R4) に 2kΩ 程度のプルアップ抵抗が必要です。
 - J1 コネクタの VDD, IO_VDD の両方に 3.3V を供給してください。あるいは、R7 をショートして VDD に 3.3V を供給してください。
 - MM-FT232HC の供給電圧はデフォルトで 5V です。KXTJ3-1057-EVK-001 に 3.3V を供給するには、MM-FT232HC の JP2 を 3.3V 側に変更してください。
 - デバイスアドレスの確認。R5 にプルアップ抵抗が接続されていることを確認。あるいは J1 コネクタの ADDR ピンを IO_VDD にプルアップ。R9 には抵抗を接続しないでください。

KXTJ3-1057-EVK-001



J1 Pin#	Description
1	VDD
2	nCS
3	-
4	-
5	SCLK/SCL
6	IO_VDD
7	SDI/SDA
8	GND
9	SDO/ADDR
10	TRIG (Connect to GND when not using external trigger option)
11	INT1
12	INT2
13	-
14	-

2.4. フォルダ構成

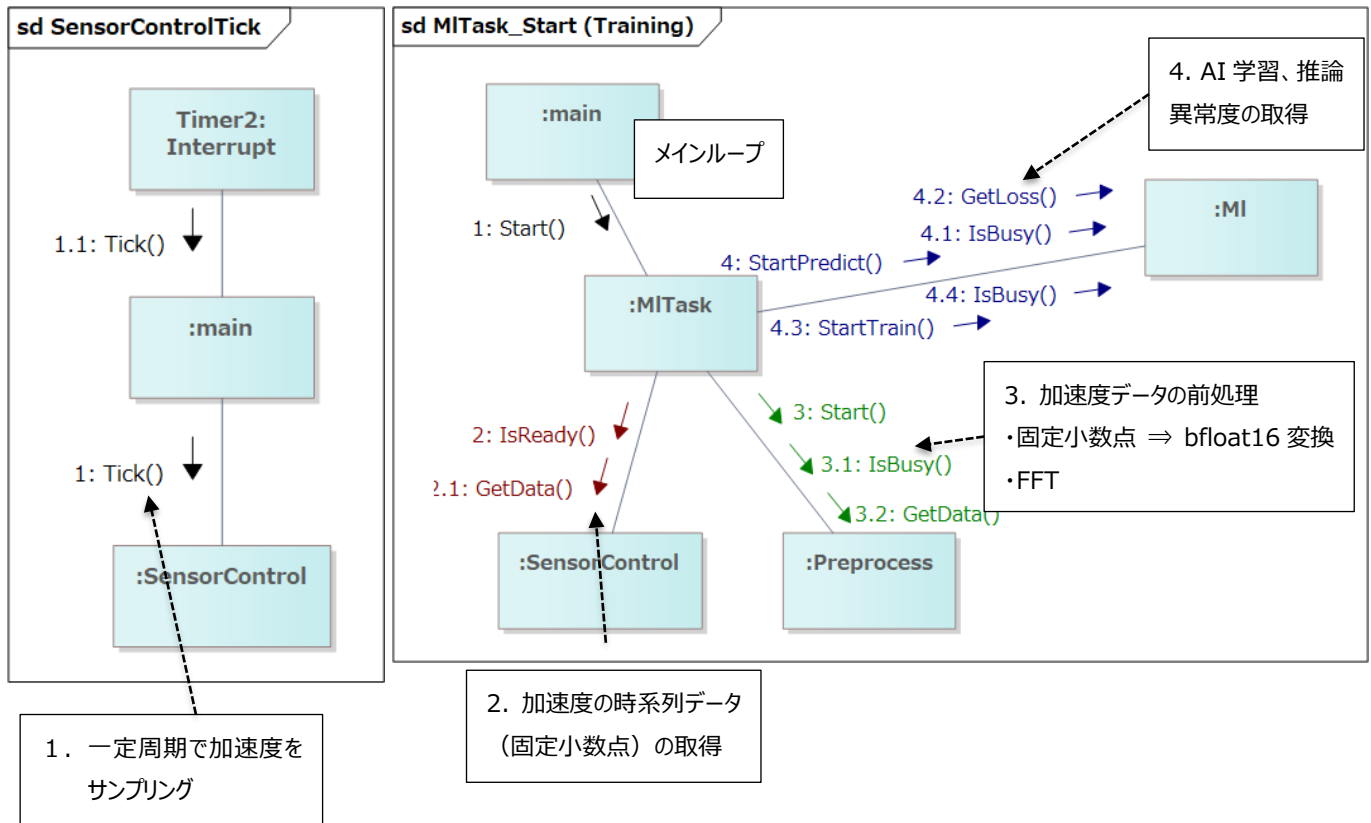
本サンプルファームウェアは、リファレンスソフトウェアに含まれるデバイスドライバを使用します。
ロームのマイコン Web サイトより、「ML63Q2500 シリーズ リファレンスソフトウェア」をダウンロードし、以下のように配置してください。

Folder	Description	
driver/inc driver/src	Peripheral Drivers	リファレンスソフトウェアに含まれる
driver/lib	Solist-AI library	本サンプルファームウェアのパッケージに含まれる
samples/Solist-AI/AnomalyDetectionDemo	本サンプルファームウェア	
hal	Sensor drivers	
sensor		

2.5. サンプルファームウェア仕様

2.5.1. 概略フロー

3 軸加速度センサから加速度をサンプリングし、一定の長さを取得後、加速度の時系列データを FFT で片側振幅スペクトルに変換し、Solist-AI™に入力して学習・推論します。



2.5.2. 異常検知

AnomalyDetector.c で、異常の判定をしています。

本サンプルファームウェアでは、単純に異常度が平均値の一定%以上の場合、異常と判定します。

2.5.3. 状態遷移

Solist-AI™ Scope から、学習・推論の状態を制御することができます。

学習・推論状態

Event / State	Stop	Training	Predict
TRAIN	Train		
PREDICT	Predict		
STOP		Stop	Stop

2.5.4. LED

学習・推論状態、異常検知の状態に応じて、LED を ON/OFF します。

	Training State				Anomaly state	
	Stop	Training	Training completed	Predict	Not detected	Detected
LED1	–	●	O	–		
LED2	–	–	–	O		
LED3					–	O

●: 点滅, O: 点灯, –: 消灯

2.5.5. センサの設定

board/SensorSettings.h で、使用するセンサを設定しています。

デフォルトでは、KXTJ3-1057 (KXTJ3-1057-EVK-001 ボード)となっています。使用するボードに合わせてセンサを変更してください。

新規にセンサを追加する場合は、初期化、Start、Stop、センサ値の取得の 4 つの関数を含むセンサのドライバを用意し、SensorSettings.h で以下のように指定してください。

```
#define SENSOR1_Init() Kxtj3Init(KXTJ3_ADDR_HIGH) // for KXTJ3-1057-EVK-001
#define SENSOR1_Start() Kxtj3Start()
#define SENSOR1_Stop() Kxtj3Stop()
#define SENSOR1_GetValue(values) Kxtj3GetRawValue(values)
```

- KXTJ3-1057-EVK-002 の場合、以下のように I2C の Device Address を変更してください。

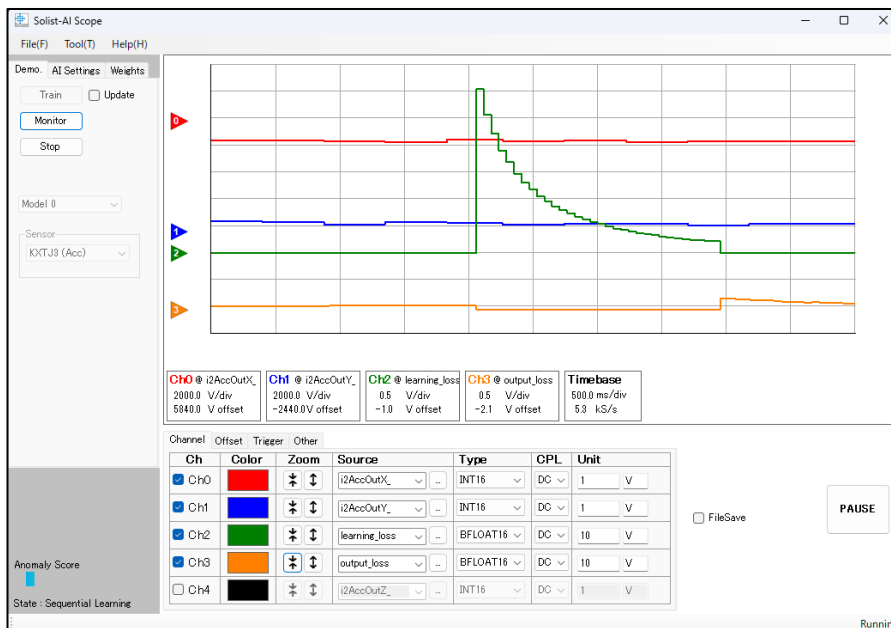
```
#define SENSOR1_Init() Kxtj3Init(KXTJ3_ADDR_LOW) // for KXTJ3-1057-EVK-002 (JP1 open (default))
```

センサのサンプリング周波数は、board/RB_D63Q25.c で定義しています。（センサ自体のデータレートは別に設定してください）

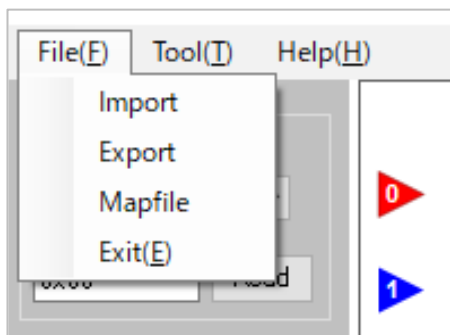
```
#define TIMER2_SAMPLING_FREQUENCY 800U ///< Sensor sampling frequency [Hz]
```

2.6. Solist-AI™ Scope を使った学習・推論の確認

Solist-AI™ Scope を使うと、リアルタイムに AI の学習状況を確認したり、推論結果をモニタしたりすることができます。



1. Solist-AI™ Scope 起動後、サンプルファームウェアの map ファイルを指定
一度指定しておけば、次回起動時も指定した map ファイルを読み込みます。
LEXIDE-Ωであれば、ビルド後 Debug フォルダ内に map ファイルが生成されます。



2. チャートに表示する信号の選択と設定
Source で信号を選択し、Type で信号のデータ型を指定。

加速度センサの X/Y/Z 軸の値 : I2AccOutX_ / I2AccOutY_ / I2AccOutZ_ (INT16)

学習時の誤差 : learning_loss (BFLOAT16)

異常度 : output_loss (BFLOAT16)

加速度センサの Zoom は 2000~5000, 誤差と異常度の Zoom は 0.01 以下に調整すると見やすくなります。

Ch0 @ i2AccOutX_ 2000.0 /div 6000.0 offset	Ch1 @ i2AccOutY_ 2000.0 /div -1040.0 offset	Ch2 @ learning_loss 0.001 /div -0.002 offset	Ch3 @ output_loss 0.001 /div -0.004 offset	Timebase 500.0 ms/div 5.6 kS/s
---	--	---	---	---

Channel	Offset	Trigger	Other
☒ Ch0			
☒ Ch1			
☒ Ch2			
☒ Ch3			

Ch	Color	Zoom	Source	Type	CPL	Gain
☒ Ch0		⌕ ↕	i2AccOutX_ ...	INT16	DC	1
☒ Ch1		⌕ ↕	i2AccOutY_ ...	INT16	DC	1
☒ Ch2		⌕ ↕	learning_loss ...	BFLOAT16	DC	1
☒ Ch3		⌕ ↕	output_loss ...	BFLOAT16	DC	1

Other タブで Timebase が 500ms/div となるように Zoom を調整し、"Roll Mode"を"Auto"に設定すると見やすくなります。

File メニュー – Export/Import で、これらの設定をファイルに保存/復元することができます。

Ch0 @ i2AccOutX_ 2000.0 V/div 6040.0 V offset	Ch1 @ i2AccOutY_ 2000.0 V/div -2440.0V offset	Ch2 @ learning_loss 0.1 V/div -0.2 V offset	Ch3 @ output_loss 0.1 V/div -0.4 V offset	Timebase 500.0 ms/div 5.4 kS/s
--	--	--	--	---

Channel	Offset	Trigger	Other
Other Option Timebase Zoom ⌕ ↕ Average Off Envelope Off Cursor Off Roll Mode Auto ☐ Show Statistics			

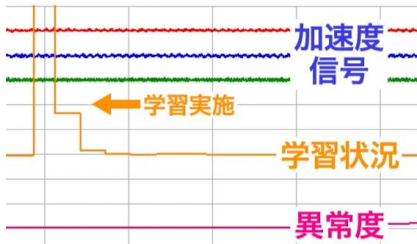
3. 学習開始ボタンを押すと初期学習を開始します。

Train

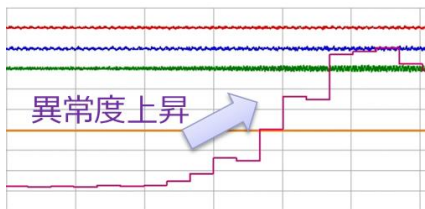
Monitor

Stop

4. 一定回数、学習すると初期学習を完了します。学習誤差（learning_loss）がゼロに収束していれば、正しく学習できています。初期学習が完了すると、逐次学習（+推論）モードに移行します。逐次学習時、異常度が大きいデータは外れ値として除外し、学習しません。外れ値と判断する異常度のしきい値は、設定タブの逐次学習閾値で設定できます。



5. モニタ開始ボタンを押すと、異常検知（推論）モードに移行します。正しく学習ができていれば、加速度センサに振動を与えると、異常度(output_loss)が大きくなります



2.7. 加速度センサの固定

加速度センサで振動データを取得する場合、測定対象にしっかりと固定してください。

固定が不十分だと、有効な振動データをとることができません。

センサの固定方法としては、以下のような方法があります。

- ・ネジ止め
- ・接着
- ・マグネットによる固定

2.8. Demo board library

サンプルファームウェアで使用している、ボードのライブラリ仕様。

2.8.1. API 仕様

Name	Summary
Board_Init(void)	MCU のペリフェラルの初期化
Board_SystemInit(void)	MCU のクロック初期化。SystemInit から実行する
LEDx_Set(bool state)	LEDx の状態をセット
LEDx_Toggle(void)	LEDx の状態を反転

2.9. Sensor Driver library

2.9.1. API 仕様

x=Kxtj3/Kx132acr など

Function	Summary
x_Init(void)	センサの初期化
x_Start(void)	センサの動作開始
x_Stop(void)	センサの動作停止（省電力モード）
x_GetRawValue(int16_t raw[])	加速度センサの出力値を取得
x_GetValue(int16_t acc[])	加速度センサの値を G 単位で取得

3. Solist-AI™ library

Solist-AI™ Scope library の仕様については、Solist-AI™ Scope User's Guide を参照してください。

ライブラリ一覧。必要なライブラリをリンクして使用してください

Library name	Summery
SolistAi_Library	Solist-AI™ library
SolistAi_Scope	Solist-AI™ Scope 用 library

3.1. AI library

Solist-AI™ アクセラレータを 3 層ニューラルネットワークの構成で使用します。

3.1.1. AI への入出力データ数

入出力データ数、隠れ層データ数の上限は、AI モデルが使用するメモリ量に依存します。以下を目安として下さい。

ML63Q25 で利用可能なパラメータの組み合わせ

modelCount	outputSize/inputSize	hiddenSize
1	512	64
1	160	128
1	4000	10
2	256	64
4	128	64

3.1.2. AI モデルの設定

ODL_Parameters を ODL_Initialize に渡すことで初期設定します。その後、ODL_Reset で重みデータを初期化します。
入出力ノード数は用途に合わせ、それ以外はまず Default 値を目安として使用下さい。

ODL_Parameters

Variable	Type	Default value	Summary
inputSize	uint16_t	96	AI (3 層ニューラルネットワーク)の入力ノード数 サンプルでは、FFT の出力データ数に合わせています
hiddenSize	uint16_t	12	AI (3 層ニューラルネットワーク)の隠れ層のノード数 異常検知の場合、入力ノードの 1/8~1/4 が目安。
outputSize	uint16_t	96	AI (3 層ニューラルネットワーク)の出力ノード数 異常検知ではオートエンコーダとして使うため、入力ノード数と同じにしてください
activationFunction	uint8_t	1	活性化関数。隠れ層を計算する際に使用する。 ODL_ACTV_LINEAR (0): Linear function. $f(x) = x$. ODL_ACTV_SIGMOID (1): Hard Sigmoid function. $f(x) = \max(0, \min(1, 0.2x+0.5))$ ODL_ACTV_RELU (2): ReLU function. $f(x) = \max(0, x)$ ODL_ACTV_TANH (3): Hard tanh. $f(x) = \max(-1, \min(1, x))$
lossFunction	uint8_t	1	損失関数。loss の計算方法。学習には影響しません。 ODL_LOSS_MAE (0): Mean Absolute Error ODL_LOSS_MSE (1): Mean Squared Error
forgettingFactor	bfloat16	0x3F80 (1.0)	逐次学習時の忘却率 (bfloat16 で、範囲は 0.5~2)。初期値は 1.0。 0 に近いほど、新しいデータを優先する
seed	uint16_t	1	重み alpha を生成する乱数のシード値 (範囲 1~32767)
scaleAlpha	bfloat16	0	重み alpha は $[-scaleAlpha, scaleAlpha]$ の一様乱数となる
scaleGamma	bfloat16	0	ESN の隠れ層から隠れ層へのフィードバック重み γ は、 $[-scaleGamma, scaleGamma]$ の一様乱数となる。scaleGamma=0 なら、フィードバックのない通常の 3 層ニューラルネットワーク
leakRate	bfloat16	0	ESN の Leak rate (0 ~ 1)
l2Param	bfloat16	0	正則項。 $10^{-3} \sim 10^3$ 程度の範囲で使用してください。 0 なら正則化は無効

本サンプルファームウェアは、Solist-AI™ Scope の設定タブから設定することもできます。

Number of input data

Nodes in hidden layer

Number of AI models

Activation function

Loss function

Forgetting factor

Threshold in seq. learning

3.1.3. 主な API の仕様

Name	Summery
ODL_Initialize	AI モデルの初期化。 入力データ数などを指定する。 モデルのサイズがデバイスの上限を超えると、エラーを返します。
ODL_StartTrain	学習開始。入力データは bfloat16 形式 異常検知の時、期待値データに入力データそのものを指定
ODL_StartPredict	推論開始。入力データは bfloat16 形式 異常検知の時、期待値データに入力データそのものを指定 教師あり学習時、検証用に推論誤差が必要なら、正解データも指定する
ODL_IsBusy	学習や推論開始後、実行中かどうか
ODL_GetLoss	Loss の取得。 学習時は学習誤差、推論時は推論誤差（異常度）
ODL_GetResult	AI の出力を取得
ODL_Reset	学習のリセット
ODL_SetModelCount	使用するモデル数を設定。デフォルトは 1
ODL_ToBfloat16	固定小数点から bfloat16 への変換
ODL_Bfloat16ToFloat	bfloat16 から float への変換
ODL_Bfloat16ToFixed	bfloat16 から固定小数点への変換
ODL_GetWeightBeta	重み Beta を取得する
ODL_GetWeightP	重み P を取得する
ODL_SetWeightBeta	重み Beta を設定する
ODL_SetWeightP	重み P を設定する
ODL_ResetReservoir	リザーバー（隠れ層）の状態をゼロクリア
ODL_GetReservoir	リザーバーの状態の取得
ODL_SetReservoir	リザーバーの状態の設定

3.1.4. API 仕様

void ODL_Bfloat16ToFixed (int16_t * y, const bfloat16 * x, uint8_t qFormat, uint16_t size)

Convert bfloat16 to fixed-point.

Parameters

out	<i>y</i>	Output buffer (Fixed point).
in	<i>x</i>	Input buffer (bfloat16).
in	<i>qFormat</i>	Specify fixed-point format of y. For example, qFormat = 12 means Q12 or Q3.12. (5.5 is 0b101.1000 0000 0000 in Q12).
in	<i>size</i>	Size of buffer.

void ODL_Bfloat16ToFloat (float * y, const bfloat16 * x, uint16_t size)

Convert bfloat16 to float.

Parameters

out	<i>y</i>	Output buffer (float).
in	<i>x</i>	Input buffer (bfloat16).
in	<i>size</i>	Size of buffer.

void ODL_GenerateRandomNumber (bfloat16 * y, uint16_t size, uint16_t seed)

Generate random numbers.

Parameters

out	<i>y</i>	Output buffer (bfloat16).
in	<i>size</i>	Number of random number generated.
in	<i>seed</i>	Seed of random number generator.

Returns

None

uint32_t ODL_GetInputAddress (void)

Get address of input data to AI accelerator for DMA.

Returns

Address of input data.

bfloat16 ODL_GetLoss (uint8_t instance)

Get loss.

Learning error during learning. Prediction error during prediction.

Parameters

in	<i>instance</i>	Index of AI model.
----	-----------------	--------------------

Returns

Value of loss.

float ODL_GetLossAsFloat (uint8_t *instance*)

Get loss as a float.

Learning error during learning. Prediction error during prediction.

Parameters

in	<i>instance</i>	Index of AI model.
----	-----------------	--------------------

Returns

Value of loss.

uint32_t ODL_GetOutputAddress (void)

Get address of output data to AI accelerator for DMA.

Returns

Address of output data.

const ODL_Parameters* ODL_GetParameters (uint8_t *instance*)

Get paramters.

Parameters

in	<i>instance</i>	Index of AI model.
----	-----------------	--------------------

Returns

Current parameters of AI model.

void ODL_GetReservoir (uint8_t *instance*, bfloat16 * *state*)

Get reservoir state.

Parameters

in	<i>instance</i>	Index of AI model.
out	<i>state</i>	Reservoir state.

void ODL_GetResult (uint8_t *instance*, bfloat16 * *y*)

Get ODL result.

Parameters

in	<i>instance</i>	Index of AI model.
out	<i>y</i>	Output buffer.

Returns

None

void ODL_GetWeightBeta (void * *beta*, uint8_t *instance*, uint32_t *offset*, uint16_t *size*)

Get weight data (Beta) after training.

Parameters

out	<i>beta</i>	Buffer of weight data to read.
in	<i>instance</i>	Index of AI model.
in	<i>offset</i>	Offset from the start address of the instance.
in	<i>size</i>	Read size.

Returns

None

void ODL_GetWeightP (void * *p*, uint8_t *instance*, uint32_t *offset*, uint16_t *size*)

Get weight data (P) after training.

Parameters

out	<i>p</i>	Buffer of weight data to read.
in	<i>instance</i>	Index of AI model.
in	<i>offset</i>	Offset from the start address of the instance.
in	<i>size</i>	Read size.

Returns

None

bool ODL_Initialize (uint8_t *instance*, const ODL_Parameters * *parameters*)

Initialize AI model.

Call this function in order of the lowest value of the instance. If you call this function with instance=1 and then call this function with instance=0, the AI model with instance=1 will not work properly.

Parameters

in	<i>instance</i>	Index of AI model.
in	<i>parameters</i>	Parameters.

Returns

Result

bool ODL_IsBusy (void)

Is ODL busy?

Parameters

-	
---	--

Return values

0	Not busy.
<i>other than 0</i>	Busy.

Is ODL busy?

void ODL_Reset (uint8_t *instance*)

Reset training.

Parameters

in	<i>instance</i>	Index of AI model.
----	-----------------	--------------------

Returns

None

void ODL_ResetReservoir (uint8_t *instance*)

Reset the reservoir state (hidden layer) to zero.

Parameters

in	<i>instance</i>	Index of AI model.
----	-----------------	--------------------

bool ODL_SetModelCount (uint8_t *modelCount*)

Set the number of AI models to use.

Parameters

in	<i>modelCount</i>	Model count.
----	-------------------	--------------

Returns

Result

void ODL_SetReservoir (uint8_t *instance*, const bfloat16 * *state*)

Set reservoir state.

Parameters

in	<i>instance</i>	Index of AI model.
in	<i>state</i>	Reservoir state.

void ODL_SetWeightBeta (const void * *beta*, uint8_t *instance*, uint32_t *offset*, uint16_t *size*)

Set weight data (Beta).

Parameters

in	<i>beta</i>	Buffer of weight data to write.
in	<i>instance</i>	Index of AI model.
in	<i>offset</i>	Offset from the start address of the instance.
in	<i>size</i>	Write size.

Returns

None

void ODL_SetWeightP (const void * *p*, uint8_t *instance*, uint32_t *offset*, uint16_t *size*)

Set weight data (P).

Parameters

in	<i>p</i>	Buffer of weight data to write.
----	----------	---------------------------------

in	<i>instance</i>	Index of AI model.
in	<i>offset</i>	Offset from the start address of the instance.
in	<i>size</i>	Write size.

Returns

None

void ODL_StartPredict (uint8_t *instance*, const bfloat16 * *x*, const bfloat16 * *t*)

Start prediction.

Parameters

in	<i>instance</i>	Index of AI model.
in	<i>x</i>	Buffer of input data. NULL means input data has been already filled by DMA.
in	<i>t</i>	Buffer of expected data to compute loss. It is optional for supervised learning. <i>x</i> is used for expected data if <i>t</i> == NULL.

Returns

None

void ODL_StartTrain (uint8_t *instance*, const bfloat16 * *x*, const bfloat16 * *t*)

Start sequential training.

Parameters

in	<i>instance</i>	Index of AI model.
in	<i>x</i>	Buffer of input data. NULL means input data has been already filled by DMA.
in	<i>t</i>	Buffer of expected data. <i>x</i> is used for expected data if <i>t</i> == NULL.

Returns

None

void ODL_ToBfloat16 (bfloat16 * *y*, const int16_t * *x*, uint8_t *qFormat*, uint16_t *size*)

Convert fixed-point to bfloat16.

Parameters

out	<i>y</i>	Output buffer (bfloat16).
in	<i>x</i>	Input buffer (Fixed point).
in	<i>qFormat</i>	Specify fixed-point format of <i>x</i> . For example, <i>qFormat</i> = 12 means Q12 or Q3.12. (5.5 is 0b101.1000 0000 0000 in Q12).
in	<i>size</i>	Size of buffer.

Returns

None

bfloat16 OSUAD_GetLoss (void)

Get the max value of loss for each model.

Learning error during learning. Prediction error during prediction.

Parameters

-	
---	--

Returns

Value of loss.

bool OSUAD_Initialize (const ODL_Parameters * *parameters*, uint8_t *numberOfInstance*)

Initialize AI models.

Parameters

in	<i>parameters</i>	Parameters.
in	<i>numberOfInstance</i>	Number of AI models.

Returns

Result

void OSUAD_Reset (void)

Reset training.

Returns

None

3.1.5. Solist-AI™ Sim で作成したモデルのインポート

Solist-AI™ Sim で保存したモデル、model1.h をインクルードし、Solist-AI™ ライブラリにモデルを設定して使用します。

model1.h

```
#define MODEL1_SEED 1 // Seed of alpha
#define MODEL1_NI 96 // Number of input data
#define MODEL1_M 12 // Number of hidden nodes
#define MODEL1_NO 96 // Number of output data
#define MODEL1_ACTIVATION 1 // Activation function
#define MODEL1_LOSS 1 // Loss function

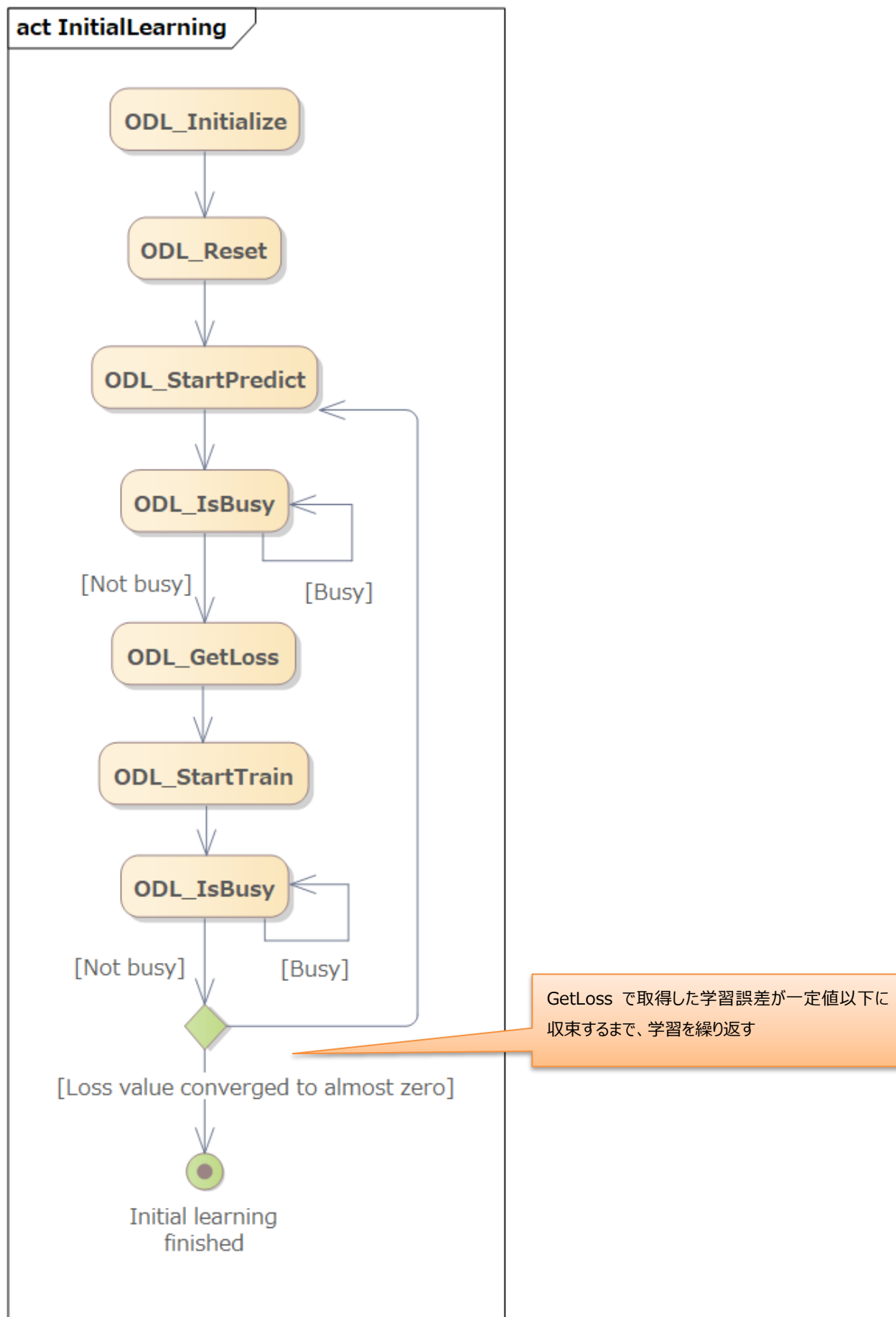
// Weights Beta & P
static const int16_t s_model1_beta[] = {...}
static const int16_t s_model1_p[] = {...}
```

Solist-AI™ ライブラリでのモデル設定

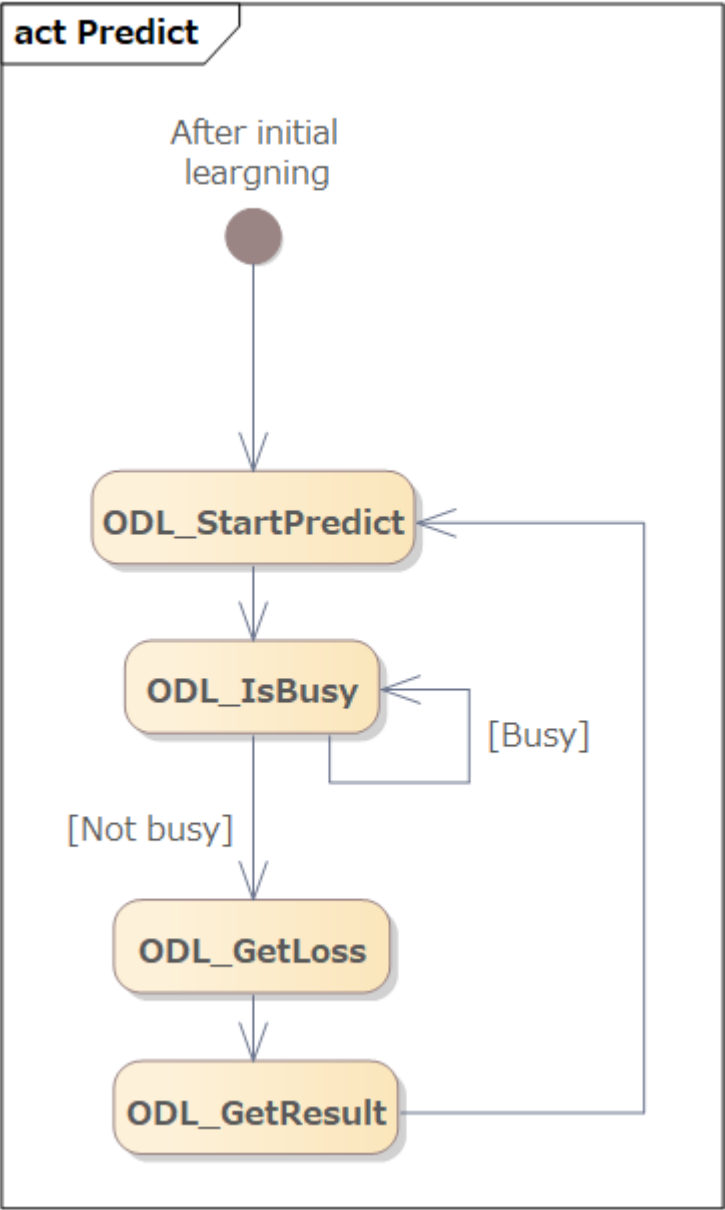
```
// 3-layer neural network
ODL_Parameters parameters =
{
    .inputSize = MODEL1_NI, // Number of nodes in input layer */
    .hiddenSize = MODEL1_M, // Number of nodes in hidden layer */
    .outputSize = MODEL1_NO, // Number of nodes in output layer */
    .activationFunction = MODEL1_ACTIVATION, /* Sigmoid function */
    .lossFunction = MODEL1_LOSS, /* Mean squared error */
    .forgettingFactor = BF_2N(0), /* Forgetting factor: 1.0 */
    .seed = MODEL1_SEED, /* Seed of weight Alpha to generate random number */
};
ODL_Initialize(0, &parameters);

// Load model
for (uint16_t i = 0; i < MODEL1_M; i++)
{
    ODL_SetWeightBeta(&s_model1_beta[i*MODEL1_NO], 0, i*MODEL1_NO*2, MODEL1_NO*2);
    ODL_SetWeightP(&s_model1_p[i*MODEL1_M], 0, i*MODEL1_M*2, MODEL1_M*2);
}
```

3.1.6. 初期学習時のフロー

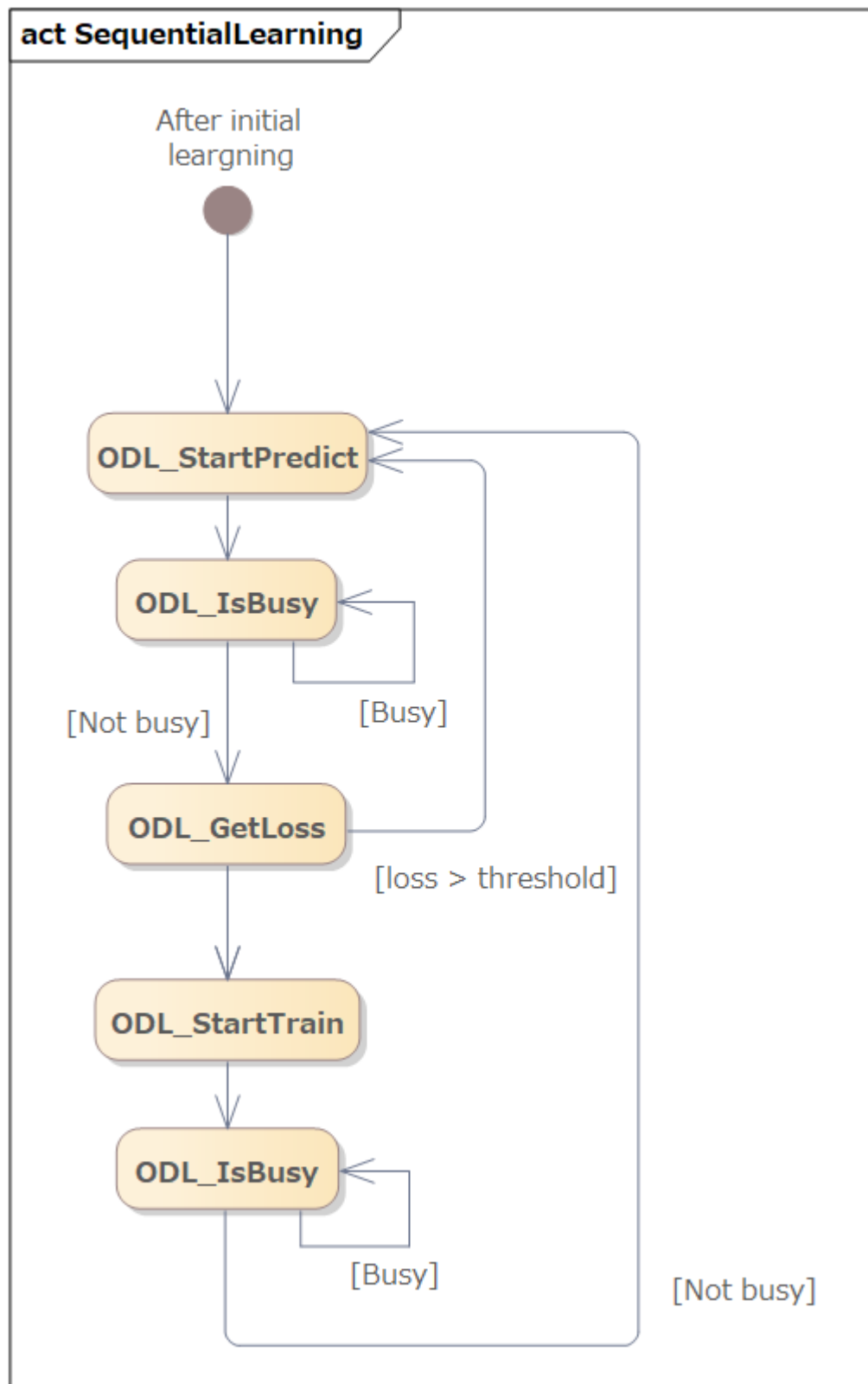


3.1.7. 推論フロー

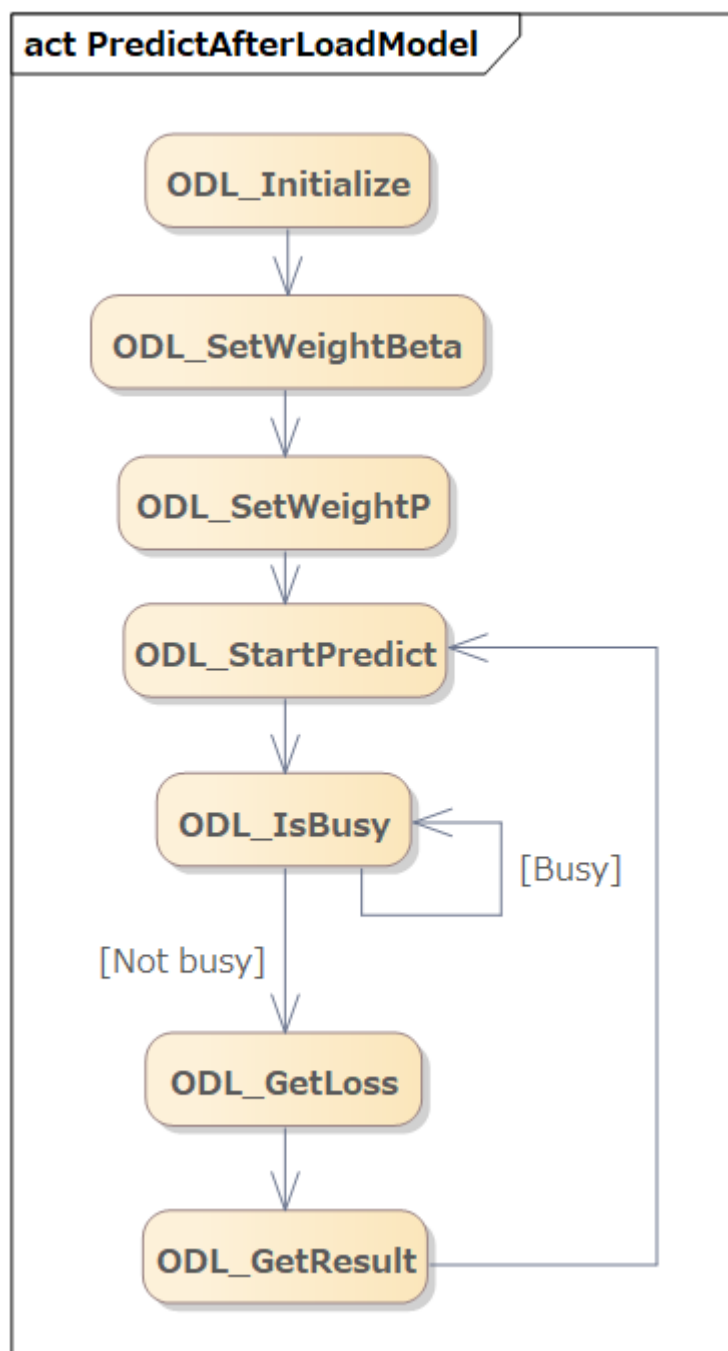


3.1.8. 逐次学習フロー

異常値、外れ値を学習しないようにするため、推論後の loss 値がある threshold 以下の時だけ学習するようにします。



3.1.9. モデルをインポート時の推論フロー



3.2. FFT library

Solist-AI™ アクセラレータを FFT の構成で使します。

3.2.1. 仕様

- FFT の出力データは片側振幅スペクトル相当になります。
FFT 出力の実部を r 、虚数部を i として、FFT の出力データ $y = |r| + |i|$
入力データ数が 256 の場合、出力データ数は 129 となります。FFT の出力データの先頭は DC 成分
- FFT の入出力データは bfloat16 形式。ODL_Tofloat16 を使って、事前に入力データを固定小数点から bfloat16 に変換してください。
- FFT の出力データ数が多すぎる場合、AI に入力する前に、FFT の出力を移動平均することで、サンプリング周波数を落とさずに、データ数を 1/2, 1/4 に減らすことができます。

Name	Summery
fft_Init	FFT の初期化。 入力データ数、窓関数を指定する。最大入力データ数は 2048 入力データ数は 2 のべき乗でなければならない
fft_Start	FFT 開始。入力データは bfloat16 形式
fft_GetResult	FFT 出力データの取得
fft_IsBusy	FFT 実行中かどうか

3.2.2. API 仕様

void fft_GetResult (fft_t yout[], uint16_t size)

Get FFT result in Single-Sided Amplitude Spectrum.

Parameters

out	yout	Output buffer (bfloat16)
in	size	Size of output buffer (N/2+1)

Returns

None

void fft_Init (uint16_t size, FftWindow window)

Build sequence for FFT.

Parameters

in	size	Input size of FFT (N). size must be a power of 2 and less than or equal to 1024.
in	window	Window function.

Returns

None

FftWindow

FFT_WINDOW_NONE	No window functions
FFT_WINDOW_HANN	Hanning Window Function

bool fft_IsBusy (void)

Is FFT busy?

Parameters

-	
---	--

Return values

0	Not busy
other than 0	Busy

void fft_Start (fft_t x[], uint16_t size)

Start fast fourier transform.

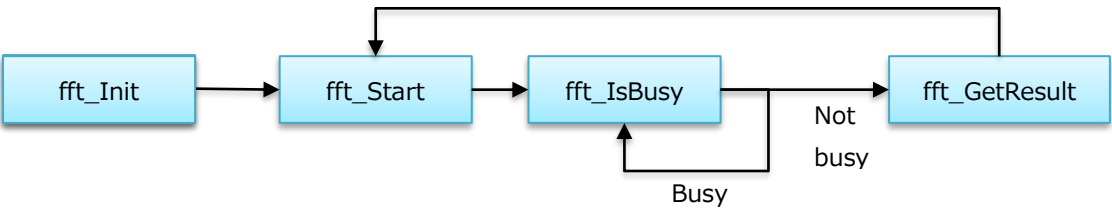
Parameters

in	x	Input buffer (bfloat16). NULL means input data has been already filled by DMA.
in	size	Input size of FFT (N). size must be a power of 2 and less than or equal to 1024.

Returns

None

3.2.3. Usage



3.3. オンデバイス学習と重みデータ

3.3.1. 3 層ニューラルネットワーク

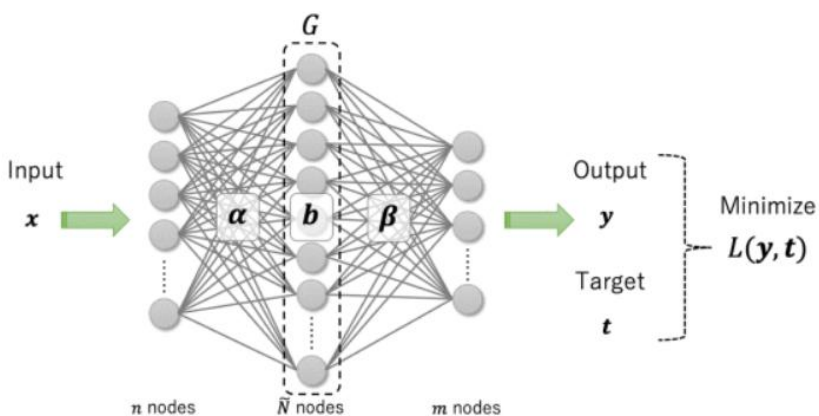
重みとは、ニューラルネットワークにおけるノード間の結合の強度を表す情報です。ニューラルネットワークが学習した情報が、重みとして蓄えられます。ニューラルネットワークの構造（入出力ノード数、隠れ層のノード数、活性化関数）と、重みの情報を保存しておけば、別の IC 上で同じ AI モデルを再現することができます。

Solist-AI™では、オンデバイス学習という方法でニューラルネットワークの学習を高速化、IC 上での学習を実現しています。

オンデバイス学習については、以下の論文を参照してください。

[A Neural Network-Based On-Device Learning Anomaly Detector for Edge Devices | IEEE Journals & Magazine | IEEE Xplore](#)

上記論文 Fig.3



Alpha (α) は 3 層ニューラルネットワークの前段の重みです。オンデバイス学習では乱数値で固定となります。

Beta (β) は 3 層ニューラルネットワークの後段の重みです。学習時に Beta を更新するために、追加で重み P が必要となります。

3.3.2. 異常検知

異常検知では、AI は入力データと出力データが一致するように学習します（オートエンコーダを使った、半教師なし学習）。

AI は正常データだけを学習します。AI は入力データと出力データの違いを異常度（推論誤差）として出力します。

3.3.3. 重みデータのメモリ使用量

学習・推論に必要なメモリ量、および復元用に AI モデルを外部に保存する際に必要なメモリ量を下表にまとめます。

AI モデルを保存する際は、ニューラルネットワークの構造（入出力ノード数、隠れ層のノード数、活性化関数）、Alpha を生成する乱数のシード値、Beta と P を保存してください。Solist-AI™ Sim で作成したモデルのインポートも参考にしてください。

	P	Beta	その他	計 [Byte]
学習・推論に必要なメモリ量	$M \times M \times 2$	$N \times M \times 2$	$M \times M \times 2 + 16 \times 1024$	$(N + M \times 2) \times M \times 2 + 16 \times 1024$
保存が必要なメモリ量 (追加学習・推論用)	$M \times M \times 2$	$N \times M \times 2$	0	$(N + M) \times M \times 2$
保存が必要なメモリ量 (推論のみ)	0	$N \times M \times 2$	0	$N \times M \times 2$

(N: 3 層ニューラルネットワークの出力データ数、M: 3 層ニューラルネットワークの隠れ層のノード数)

3.3.4. ESN (Echo State Network)

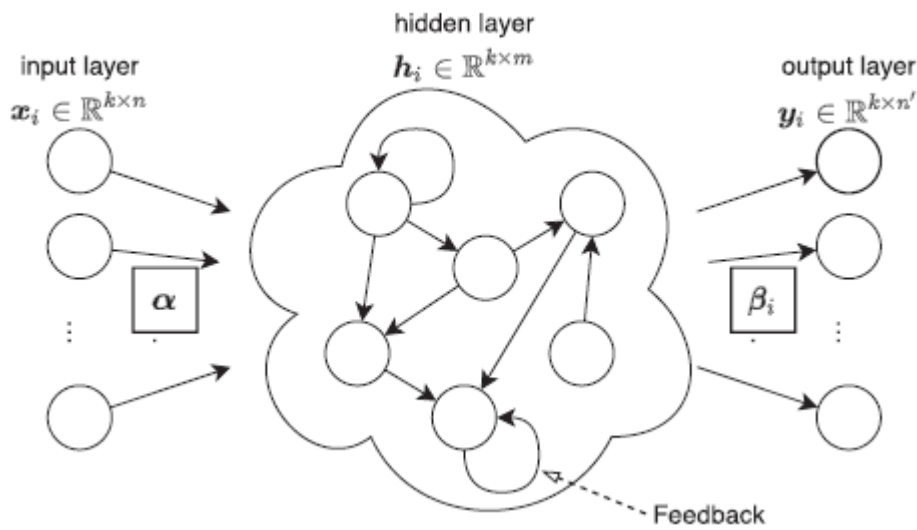


Fig. 2 Echo state network

[An Area-Efficient Recurrent Neural Network Core for Unsupervised Time-Series Anomaly Detection \(jst.go.jp\)](https://jst.go.jp)

リサバーコンピューティングで使われる ESN (Echo State Network)は、RNN (Recurrent Neural Network)の一種です。ESN は隠れ層から隠れ層へのフィードバックを持つ 3 層ニューラルネットワークとして表現することができ、Solist-AI library から利用可能です。

x_i : 入力, h_i : 隠れ層 ($h_i = (1 - \delta)h_{i-1} + \delta G(x_i\alpha + h_{i-1}\gamma)$), y_i : 出力 ($y_i = h_i\beta$)

G : 隠れ層の活性化関数、 δ : Leak rate (漏れ率忘却率)

α : 入力層から隠れ層への重み、 γ : 隠れ層から隠れ層へのフィードバックの重み、 β : 隠れ層から出力層への重み

ESN 使用時も、保存が必要な重みデータは α と β のみです。 α 、 γ は乱数値となります。

3.3.5. bfloat16

Solist-AI™のアクセラレータは bfloat16 で演算します。

float16 は 16bit の浮動小数点フォーマット。

32-bit float から bfloat16 に変換する場合、下位 16bit を切り捨ててください。

bfloat16 から 32-bit float に変換する場合、bfloat16 の 16bit をそのまま上位 16bit とし、下位 16bit はゼロとしてください。

Example) 1.23

Decimal	bfloat16 (Hex.)	32-bit float (Hex.)
1.23	0x3F9D	0x3F9D70A4

bfloat16

	Sign	Exponent (8bit)								Fraction (7bit)						
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

IEEE754 32-bit float

	Sign	Exponent (8bit)								Fraction (23bit)										
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	...	1	0

Example) bfloat16 から float への変換

```
bfloat16 b = 0x3f80; // 1.0
uint32_t t = (uint32_t)b << 16;
float f = *(float*)&t;
```

4. 商標について

Arm®, Keil®, ULINK2™および μVision®は、米国および／またはその他の地域における Arm Limited（またはその子会社・関連会社）の登録商標または商標です。

「Solist-AI™」は、ローム株式会社の商標または登録商標です。

ご注意

- 1) 本資料に記載されているソフトウェア及びシステム（以下あわせて「本ソフトウェア」といいます）に関する情報、及び本資料の記載内容は、本資料発行時点のものであり、予告なく変更することがあります。
- 2) 本製品又は本ソフトウェアを、極めて高度な信頼性が要求され、その故障や誤動作が人の生命、身体への危険若しくは損害、又はその他の重大な損害の発生に関わるような機器又は装置（医療機器、輸送機器、交通機器、航空宇宙機器、原子力制御装置、燃料制御、カーアクセサリを含む車載機器、各種安全装置等）（以下「特定用途」といいます）へのご使用を検討される際は事前にローム営業窓口までご相談くださいようお願い致します。ロームの文書による事前の承諾を得ることなく、特定用途に使用したことによりお客様又は第三者に生じた損害等に関し、ロームは一切その責任を負いません。
- 3) ロームは常に品質・信頼性の向上に取り組んでいますが、半導体製品は種々の要因で故障・誤作動する可能性があります。万一、本ソフトウェアと使用するローム製品（以下「本製品」といいます）が故障・誤作動した場合であっても、その影響により人身事故、火災損害等が起こらないようご使用機器でのデレーティング、冗長設計、延焼防止、バックアップ、フェイルセーフ等の安全確保をお願いします。定格を超えたご使用や使用上の注意事項が守られていない場合、ロームは一切その責任を負いません。
- 4) ロームは、本ソフトウェアの使用または使用不能によってお客様に生じるいかなる直接的、間接的損害(情報、データ、プログラム等の無体財産の損失、利益の喪失、中断などによる損害を含む)に関して、一切の責任を負いません。
- 5) 本ソフトウェア、本資料及び本製品には、ロームに帰属する機密情報、技術情報又はトレードシークレットが記載されておりますので、ロームの文書による事前の承諾を得ることなく、その全部又は一部について以下の行為をすることは固くお断り致します。
 - (ア) 第三者へ開示すること
 - (イ) 逆アセンブル、リバースエンジニアリングその他の解析をすること
 - (ウ) 転載、複写、複製すること
 - (エ) 本ソフトウェアに含まれている著作権表示を取り除くこと
- 6) 本製品、本ソフトウェア及び本資料に記載の技術を輸出または国外へ提供する際には、「外国為替及び外国貿易法」、「米国輸出管理規則」など適用される輸出関連法令を遵守し、それらの定めにしたがって必要な手続きを行ってください。
- 7) ロームは、本ソフトウェア及び本資料に記載された情報に関し第三者の知的財産権及びその他の権利について権利侵害がないことを保証するものではありません。従いまして、上記権利侵害に関し、ロームは一切その責任を負いません。
- 8) ロームは、本資料に記載された情報についてローム若しくは第三者が所有又は管理している知的財産権その他の権利の実施又は利用を、本資料によって、明示的にも黙示的にも、お客様に許諾するものではありません。
- 9) お客様は、（１）お客様による第三者の権利の侵害（オープンソースソフトウェアプログラムのライセンス条件への違反を含むがこれに限定されない）や法令違反により生じる、または、（２）お客様が本資料に違反することやこれに関連して生じる賠償金等の請求による損失、罰金や費用（弁護士費用を含むがこれに限定されない）から、ローム及びロームの役員や従業員を免責することを保証し、損害を与えないようにし、また保護するものとします。
- 10) 本資料は、正確を期して慎重に作成したものです。誤りがないことを保証するものではありません。万一、本資料に記載された内容の誤りに起因してお客様又は第三者に損害が生じた場合であっても、ロームは一切その責任を負いません。



ローム製品のご利用ありがとうございます。

製品に関するお問い合わせは、下記よりお願い致します。

ROHM Customer Support System

<http://www.rohm.co.jp/contact/>